

FRED TALKS

29th September 2026

CONTENTS

The new rules of context engineering for Claude 5 generation models

CLAUDE.COM · 1148 WORDS

Mastering Codex Remote for engineering

DEVELOPERS.OPENAI.COM · 2207 WORDS

Scaling Managed Agents: Decoupling the brain from the hands

ANTHROPIC ENGINEERING BLOG · 2024 WORDS

Jupiter Icy Moons Explorer

XKCD.COM · 23 WORDS

The new rules of context engineering for Claude 5 generation models

CLAUDE.COM · 09 AUG 2026 · [SOURCE](#)

Then and now

There were a number of previous context engineering best practices that had become myths. Including:

Then: Give Claude rules

Now: Let Claude use judgement

When we first rolled out Claude Code, we needed to be sure that Claude avoided worst case scenarios, such as deleting files. This meant we would give particularly strong guidance that might not always be true. For example, in the system prompt we used to say:

In code: default to writing no comments. Never write multi-paragraph docstrings or multi-line comment blocks — one short line max. Don't create planning, decision, or analysis documents unless the user asks for them — work from conversation context, not intermediate files.

But for a certain subset of prompts, this guidance would be wrong. In the case of documentation, the user may have their own preferences, or specific parts of very complex code might need multi-line comment blocks.

Still, without these guardrails for older models, the comments Claude wrote would be incorrect in many cases and we had to accept this tradeoff. But newer models have better judgement and can handle these decisions well without explicit rules.

In the new system prompt we say: *Write code that reads like the surrounding code: match its comment density, naming, and idiom.*

Then: Give Claude examples

Now: Design interfaces

The number one rule for tool usage was to give Claude examples on how to use them. With our newest models, we've found that giving examples actually constrains them to a certain exploration space.

Instead of using examples, think more about the design of your tools, scripts and files- what parameters does Claude have and how can they be more expressive?

For example, in the Todo tool example, just listing status as an enumeration between pending, in_progress, and completed, hints to Claude about how to use it. The instruction on keeping one item in_in_progress helps define our requested behavior.

Then: Put it all upfront

Now: Use progressive disclosure

Because Claude Code was focused on coding, our system prompt included detailed information on how to do code review and verification. These were not always needed, but when they were, it was crucial information.

Since then, Claude Code has gotten very competent at using progressive disclosure- loading the right context at the right times. For example, we moved verification and code review into their own skills that Claude Code could selectively call.

But progressive disclosure is not just for skills, we also use it for tools. Some of our tools are 'deferred loading,' which means the agent must search for their full definitions using ToolSearch before using them. This allows us to have more tools (such as our Task tools) that don't take up context until they're needed.

The same can be applied to your own CLAUDE.md and Skill.md files. A common myth is that you want to make these a central repository for every known practice that you *might* run into, because Claude would not find it otherwise. Instead, consider having a tree of files that can be loaded at the right time.

Then: Repeat yourself

Now: Simple tool descriptions

Earlier Claude models could sometimes need repeated instructions or be more likely to listen to instructions at the end of their context window than at the start. This meant our system prompt would sometimes have references to tools in the main system prompt as well as instructions in the tool description.

We found we could delete these repeat examples and put instructions on how to use tools in the tool descriptions rather than the system prompt.

Then: Memory in CLAUDE.md files

Now: Auto-memory

We used to encourage users to save things to Claude's memory, by using the # hotkey to write to their CLAUDE.md automatically. Instead, Claude now automatically saves memories that are relevant to the work and to you.

Then: Simple specs

Now: Rich references

In plan mode, Claude Code has heavily relied on markdown files with plans. Storing these files as plans helped Claude refer to them when needed. Another similar best practice was to store specs in the codebase for Claude to refer to while working across longer projects.

But we've found that Claude can handle increasingly more complicated references. Instead of simple markdown files, Claude can reference HTML artifacts created by our new artifacts feature.

You may also give Claude references in the form of code. A spec may also be a detailed test suite, or a function in a different codebase that Claude might port.

Rubrics are another form of references. Rubrics allow Claude to try and verify your taste in a particular field (e.g. what does a good API design look like) by using dynamic workflows and spinning up verifier agents with those rubrics.

Applying this to your context

Pulling this all together, what does this look like when you assemble your context?

System Prompt

A system prompt is heavily tied to the product context. It tells Claude what product it's operating in and what it's doing. For Claude Code, you will likely never modify this, but if you are building your own agent harness, this is where you should spend a lot of time.

CLAUDE.md

Keep your CLAUDE.md lightweight and briefly describe what your repo is for, but spend most of the tokens on gotchas inside of the codebase. For example, you may organize your code to keep types in one monolithic file and nowhere else. Avoid stating 'the obvious' things Claude should know by looking at your file system or your repo.

Use progressive disclosure heavily, for example if you have several unique instructions on how to verify your work, create a verification skill and reference it from your CLAUDE.md.

Skills

Think of skills as lightweight guides to let Claude find information when needed. Avoid making them overconstrained, except in highly important areas.

For long skills, try and use progressive disclosure as much as possible- divide it into many files and split them out.

It's best when skills encode particular opinions, knowledge, or best practices that are particular to you, your team, or product.

References

You can @ mention files to include them as references. References allow Claude to refer to in-depth information about the current plan.

This might be in specs files, mockups, or even entire codebases. Generally you should prefer files that are in code as it provides clear, high-fidelity instructions to Claude in a language it knows very well. For example, a HTML mockup of a design will generally produce better results than a description of the design or a screenshot.

Try simplifying

Across your system prompt, skills, and CLAUDE.md files, you may need to simplify just like we did. We rolled out a new command called ``claude doctor``, which will help you do this automatically as well. For more details on prompting more advanced models specifically, check out our [Fable field guide](#).

This article was written by Thariq Shihpar, member of technical staff, Anthropic.

Mastering Codex Remote for engineering

DEVELOPERS.OPENAI.COM · 25 JUN 2026 · [SOURCE](#)

Codex Remote is easy to underestimate.

At first glance, it looks like a way to check on a coding task from your phone. That is useful, but it misses the bigger idea. The real power of Codex Remote is that it lets you start, direct, review, and organize work running on your development machines without pretending that an iPhone should be a tiny terminal.

Over the last two months, we have added a surprising amount of depth to that loop: remote host connections, worktrees, goals, side chats, inline code review, queued and steering prompts, attachments, skills and plugins, archived threads, security controls, and a long tail of small details that make the app useful for serious work.

This is the field guide I wish every new power user had.



The right mental model: Your phone is the control plane

The code still runs where it belongs: on your Mac, Windows machine, devbox, or other connected host. Codex Remote gives you a native interface for controlling that work.

That distinction matters. The goal is not to reproduce every terminal affordance on a small screen. The goal is to make the decisions that unblock an agent easy from anywhere:

- What repository and workspace should it use?
- Should this run in the current branch or a fresh worktree?
- Should my next message wait, or should it redirect the active turn?
- Is this command safe to approve?
- What changed, and do I agree with it?
- Should this become a durable goal, a separate thread, or a quick side question?

Once you use the app this way, it stops feeling like remote desktop software and starts feeling like an engineering control plane.

1. Start the task with the right boundaries

Good agent work starts with a correctly scoped environment. Codex Remote lets you choose the connected host and workspace before the first prompt. For a new thread, you can also choose a branch, create a separate worktree, and run the environment setup associated with it.

That makes a few useful patterns possible:

- Use the current checkout for a quick investigation.
- Create a new worktree for a change that should stay isolated.
- Start from the intended base branch instead of fixing Git state later.
- Let the environment setup run before asking Codex to build or test.

This is one of the most important power-user habits: Spend 10 seconds choosing the right execution context and save 10 minutes of cleanup later.

The composer can also carry more context than plain text. You can attach files, photos, or a fresh camera capture. Skills and plugins appear inline, which makes it much easier to confirm that your prompt is invoking the capability you intended.

My rule is simple: If a screenshot, file, or named skill can remove ambiguity, attach or mention it before the first turn.

2. Learn the difference between Queue and Steer

This is probably the least obvious high-leverage setting in Codex Remote.

When Codex is already working, a follow-up can do one of two things:

- **Queue** waits until the current response finishes, then sends your prompt as the next turn.
- **Steer** injects guidance into the work already in progress.

Queue is the safe default. Use it for a second task, an extra test request, or anything that should happen after the current work.

Steer is for correcting direction while the cost of continuing down the wrong path is still growing:

- | Keep the fix inside the mobile package. Do not refactor the shared renderer.
- | The reproduction only happens after reconnect. Test the resumed path, not the live path.
- | Stop investigating the UI. Check whether the server removed the item during resume.

That turns the phone into something more useful than a status screen. You can intervene at the exact moment a run needs judgment.

You can choose the default follow-up behavior in Codex settings. I keep Queue as the default and use Steer deliberately; accidental mid-turn redirection is usually more expensive than waiting.

3. Use side chats as a branch of thought

Long-running coding threads accumulate valuable context. Interrupting one with every side question makes the main transcript noisy and can pull the agent away from the objective.

That is what side chats solve.

Use `/side` to open a lightweight conversation connected to the current thread. Use `/side <prompt>` to open it with a question already prepared. Even better, select text in the transcript and choose **Ask in side chat**. The selected passage becomes the starting context for the new conversation.

I use side chats for questions such as:

- Why did Codex choose this architecture?
- What does this error actually mean?
- Is this behavior consistent with the desktop app?
- Give me a release-note version of this implementation detail.
- What should I verify before I approve this command?

The distinction is useful: The main thread owns the work; the side chat helps me understand the work.

4. Use Plan for the path and Goal for the outcome

Plan mode and goals solve different problems.

Plan mode asks Codex to propose the implementation path before changing code. It is useful when the task is underspecified, risky, or likely to touch several systems.

A goal is durable. It tells Codex what outcome to keep pursuing across turns. On mobile, `/goal` can create and manage that objective, while progress remains visible as the work continues.

A practical pattern is:

1. Start with Plan mode for a risky change.
2. Inspect the proposed boundaries.

3. Turn the accepted outcome into a goal when the work will require multiple iterations.
4. Let Codex continue through implementation, tests, review feedback, and cleanup without restating the objective every time.

Plans answer “How should we approach this?” Goals answer “What must be true before we are done?”

5. Review code without leaving the conversation

The review loop is where Codex Remote becomes genuinely useful for engineering rather than merely convenient.

Completed turns can show a summary of changed files. From there, you can open the diff, inspect individual files, expand or collapse sections, wrap long lines, and open a source file with syntax highlighting. You can attach inline comments to the relevant lines and send that review context back to Codex.

There are several levels to the workflow:

- Open the changed-file summary for a quick sanity check.
- Tap through to the full source when a diff lacks surrounding context.
- Add inline comments for precise corrections.
- Use the review command to review local changes or compare against a branch.
- Link a file back into the chat when you want Codex to reason about that file specifically.

This makes a tight mobile review loop possible:

1. Codex completes the implementation.
2. I inspect the diff from my phone.
3. I leave two inline comments.
4. Codex addresses them in the same thread.
5. I review the smaller follow-up diff.

The important point is not that a phone replaces a large monitor for deep code reading. It does not. The point is that many reviews are blocked on one or two decisions, and those decisions no longer need to wait until I am back at my desk.

6. Treat permissions as part of the workflow

Remote work is only useful if control stays explicit.

Codex Remote surfaces approval requests for commands, file changes, network access, and connected tools. Depending on the request and host configuration, an approval may apply once, for the current chat, or more broadly.

The power-user move is not to approve everything. It is to choose the narrowest permission that keeps the work moving.

For a well-understood command in a trusted thread, a chat-scoped approval can remove repetitive interruptions. For an unfamiliar command, a sensitive repository, or a request whose effect is unclear, approve once—or deny it and ask Codex to explain or use a safer approach.

You can also choose the thread's broader approval behavior when starting work. Think of this as part of task setup, alongside the host, workspace, branch, and model.

7. Manage context before it becomes a problem

Agent threads are stateful, and long threads eventually accumulate enough context to become slower or less focused.

Codex Remote has a small set of tools for managing that lifecycle:

- `/status` shows session details, workspace, context usage, and available rate-limit information.
- The optional context indicator keeps remaining context visible in the composer.
- `/compact` compresses an overgrown thread while preserving the useful working state.
- `/fork` creates a new thread from the current one when you want shared history but a different direction.

The practical sequence is: Check status, compact when the objective is still the same, and fork when the objective has diverged.

Do not use a side chat and a fork interchangeably. A side chat is a lightweight question around the current work. A fork is a new primary line of work that inherits the thread's history.

8. Keep a clean Thread Desk

My own Codex workflow increasingly looks like a small operations desk.

I pin the few threads that are actively moving, rename them around outcomes, and archive them aggressively once the work is done. Archived-thread browsing makes this safe: Archive is organization, not deletion.

Notifications are part of that system. A completion notification can open the relevant Codex task directly, so the handoff from “agent finished” to “human reviews” is one tap.

Spotlight and Shortcuts can open Codex Remote directly. On iPad, keyboard shortcuts make the app surprisingly efficient: Create a new thread, move between threads, open changed files, pin, rename, and archive without reaching for every control.

This is the “chief of staff” use case for Codex Remote. The app is not only where I issue prompts. It is where I keep track of which pieces of engineering work are active, blocked, waiting for review, or finished.

9. The hidden command palette

Typing `/` reveals the fastest route to many of the app's advanced behaviors. Availability can depend on the connected host, app version, and account configuration.

Command	What it is for
<code>/plan</code>	Toggle Plan mode before implementation.
<code>/goal <objective></code>	Create or update a durable objective.
<code>/side [question]</code>	Ask a side question without disrupting the main thread.
<code>/review</code>	Review local changes or compare them with a branch.
<code>/status</code>	Inspect the session, workspace, context, and rate limits.
<code>/compact</code>	Compress a long thread's context.
<code>/fork</code>	Create a new primary thread from the current history.

Command	What it is for
<code>/fast</code>	Switch between standard and faster execution when available.
<code>/feedback</code>	Send product feedback tied to the current session.

The command palette is worth learning because it exposes the product’s real conceptual model: Plan, pursue, branch, review, inspect, and recover.

10. Five workflows that work especially well on mobile

The release captain

Start a focused thread for a release or pull request. Ask Codex to check the current branch, CI state, open review feedback, and release inclusion. Pin the thread. When new information arrives, steer only when it invalidates the active investigation; otherwise, queue it. Review the final diff or release note from the phone and archive the thread when the release lands.

The interrupt-driven bug fix

Attach a screenshot, log, or captured file. Ask Codex to diagnose before editing. Use a side chat to interrogate one suspicious error without derailing the main investigation. Once the cause is clear, return to the main thread and authorize the narrow fix.

The mobile reviewer

Run a review against the intended branch, inspect the changed-file summary, open the files that matter, and attach inline comments. Ask Codex to address only those comments, then review the follow-up diff.

The long-running objective

Create a goal with a concrete completion condition: Tests green, review feedback resolved, or a reproducible performance threshold met. Check progress from notifications and status rather than repeatedly asking “Are you done?” Use queued prompts for additional work and reserve steering for corrections.

The multi-machine operator

Keep hosts named clearly and organize work by machine and workspace. Use the phone to start a task on the machine that has the right checkout, credentials, simulator, or operating system. This is especially useful when one task requires a Mac and another needs a Windows host.

Small features with outsized impact

Some of my favorite additions are not headline features:

- Edit the latest sent prompt instead of adding a correction turn.
- Save or copy rendered images from a task.
- See queued prompts while Codex is still running.
- Open completed tasks directly from notifications.
- Browse archived threads instead of keeping the active list crowded.
- Wrap long diff lines when reading on a narrow screen.
- Use Face ID or the device passcode to protect Codex separately.
- See skills and plugins inline in the composer before sending.
- Capture a photo directly into a prompt when the real-world object is the context.
- Start a side chat from selected transcript text rather than re-explaining the question.

None of these changes the fundamental agent model. Together, they remove the friction that makes a remote workflow feel fragile.

The larger lesson

The best mobile software does not shrink a desktop interface. It identifies the decisions that matter when you are away from the desk and makes those decisions fast, legible, and safe.

That is how I think about Codex Remote now. It is where I can choose the right environment, set the objective, redirect a run, answer an approval, inspect the result, and keep the whole queue of engineering work coherent.

The computer still does the work. The phone keeps me in control.

Scaling Managed Agents: Decoupling the brain from the hands

ANTHROPIC ENGINEERING BLOG · 24 APR 2026 · [SOURCE](#)

Get started with Claude Managed Agents by following our [docs](#).

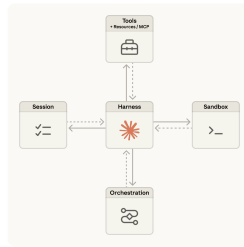
A running topic on the Engineering Blog is how to [build effective agents](#) and [design harnesses](#) for [long-running work](#). A common thread across this work is that harnesses encode assumptions about what Claude can't do on its own. However, those assumptions need to be frequently questioned because they can [go stale](#) as models improve.

As just one example, in prior work [we found](#) that Claude Sonnet 4.5 would wrap up tasks prematurely as it sensed its context limit approaching—a behavior sometimes called “context anxiety.” We addressed this by adding context resets to the harness. But when we used the same harness on Claude Opus 4.5, we found that the behavior was gone. The resets had become dead weight.

We expect harnesses to continue evolving. So we built Managed Agents: a hosted service in the Claude Platform that runs long-horizon agents on your behalf through a small set of interfaces meant to outlast any particular implementation—including the ones we run today.

Building Managed Agents meant solving an old problem in computing: how to design a system for “[programs as yet unthought of](#).” Decades ago, operating systems solved this problem by virtualizing hardware into abstractions—*process*, *file*—general enough for programs that didn't exist yet. The abstractions outlasted the hardware. The `read()` command is agnostic as to whether it's accessing a disk pack from the 1970s or a modern SSD. The abstractions on top stayed stable while the implementations underneath changed freely.

Managed Agents follow the same pattern. We virtualized the components of an agent: a session (the append-only log of everything that happened), a harness (the loop that calls Claude and routes Claude's tool calls to the relevant infrastructure), and a sandbox (an execution environment where Claude can run code and edit files). This allows the implementation of each to be swapped without disturbing the others. We're opinionated about the shape of these interfaces, not about what runs behind them.



We started by placing all agent components into a single container, which meant the session, agent harness, and sandbox all shared an environment. There were benefits to this approach, including that file edits are direct syscalls, and there were no service boundaries to design.

But by coupling everything into one container, we ran into an old infrastructure problem: we’d adopted a *pet*. In the pets-vs-cattle analogy, a pet is a named, hand-tended individual you can’t afford to lose, while cattle are interchangeable. In our case, the server became that pet; if a container failed, the session was lost. If a container was unresponsive, we had to nurse it back to health.

Nursing containers meant debugging unresponsive stuck sessions. Our only window in was the WebSocket event stream, but that couldn’t tell us *where* failures arose, which meant that a bug in the harness, a packet drop in the event stream, or a container going offline all presented the same. To figure out what went wrong, an engineer had to open a shell inside the container, but because that container often also held user data, that approach essentially meant we lacked the ability to debug.

A second issue was that the harness assumed that whatever Claude worked on lived in the container with it. When customers asked us to connect Claude to their virtual private cloud, they had to either peer their network with ours, or run our harness in their own environment. An assumption baked into the harness became a problem when we wanted to connect it to different infrastructure.

The solution we arrived at was to decouple what we thought of as the “brain” (Claude and its harness) from both the “hands” (sandboxes and tools that perform actions) and the “session” (the log of session events). Each became an interface that made few assumptions about the others, and each could fail or be replaced independently.

The harness leaves the container. Decoupling the brain from the hands meant the harness no longer lived inside the container. It called the container the way it called any other tool: `execute(name, input) → string`. The container became cattle. If the container died,

the harness caught the failure as a tool-call error and passed it back to Claude. If Claude decided to retry, a new container could be reinitialized with a standard recipe: `provision({resources})`. We no longer had to nurse failed containers back to health.

Recovering from harness failure. The harness also became cattle. Because the session log sits outside the harness, nothing in the harness needs to survive a crash. When one fails, a new one can be rebooted with `wake(sessionId)`, use `getSession(id)` to get back the event log, and resume from the last event. During the agent loop, the harness writes to the session with `emitEvent(id, event)` in order to keep a durable record of events.

Component	Interface (methods)	Satisfiability
Session	<code>getSession(session_id)</code> → {events, time}[] <code>getSession(session_id)</code> → {sessionId}[] // not yet implemented <code>setSession(id, event)</code>	Any agent only log that can be accessed at any time, any event path and enough idempotent updates → Pangloss, QDiffs, in-memory errors, etc.
Orchestration	<code>orchestrate_session(id) → void</code>	Any scheduler that can call a function with any id and only on failure → agent job is a process monitor, a scheduling lib.
Harness	<code>updateEffectivity()</code> → Effectiveness[]	Any logic that tracks effects and reports progress to the session.
Sandbox	<code>provision({resources})</code> <code>cancelSession, {event}</code> → {sessionId}	Any system that can be verified and accesses other any process as long as a log path, a session container, etc.
Resource	<code>getSession_id,</code> <code>event, path[]</code>	Any resource that can be verified and can fetch from its references → Pangloss, QDiffs, log events, etc.
Tools	<code>tools, description,</code> <code>input, output</code>	Any capability that can be a name and a log path → MCP, harness, custom tools, etc.

The security boundary. In the coupled design, any untrusted code that Claude generated was run in the same container as credentials—so a prompt injection only had to convince Claude to read its own environment. Once an attacker has those tokens, they can spawn fresh, unrestricted sessions and delegate work to them. Narrow scoping is an obvious mitigation, but this encodes an assumption about what Claude can't do with a limited token—and Claude is getting increasingly smart. The structural fix was to make sure the tokens are never reachable from the sandbox where Claude's generated code runs.

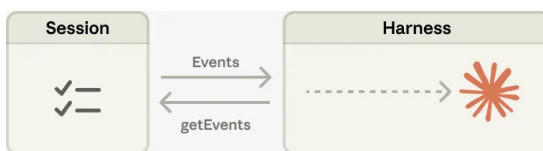
We used two patterns to ensure this. Auth can be bundled with a resource or held in a vault outside the sandbox. For Git, we use each repository's access token to clone the repo during sandbox initialization and wire it into the local git remote. Git `push` and `pull` work from inside the sandbox without the agent ever handling the token itself. For custom tools, we support MCP and store OAuth tokens in a secure vault. Claude calls MCP tools via a dedicated proxy; this proxy takes in a token associated with the session. The proxy can then fetch the corresponding credentials from the vault and make the call to the external service. The harness is never made aware of any credentials.

The session is not Claude's context window

Long-horizon tasks often exceed the length of Claude's context window, and the standard ways to address this all involve irreversible decisions about what to keep. We've explored these techniques in [prior work](#) on context engineering. For example, compaction lets Claude save a

summary of its context window and the memory tool lets Claude write context to files, enabling learning across sessions. This can be paired with context trimming, which selectively removes tokens such as old tool results or thinking blocks.

But irreversible decisions to selectively retain or discard context can lead to failures. It is difficult to know which tokens the future turns will need. If messages are transformed by a compaction step, the harness removes compacted messages from Claude's context window, and these are recoverable only if they are stored. Prior work has explored ways to address this by storing context as an object that lives *outside* the context window. For example, context can be an object in a REPL that the LLM programmatically accesses by writing code to filter or slice it.



In Managed Agents, the session provides this same benefit, serving as a context object that lives outside Claude's context window. But rather than be stored within the sandbox or REPL, context is durably stored in the session log. The interface, `getEvents()`, allows the brain to interrogate context by selecting positional slices of the event stream. The interface can be used flexibly, allowing the brain to pick up from wherever it last stopped reading, rewinding a few events before a specific moment to see the lead up, or rereading context before a specific action.

Any fetched events can also be transformed in the harness before being passed to Claude's context window. These transformations can be whatever the harness encodes, including context organization to achieve a high prompt cache hit rate and context engineering. We separated the concerns of recoverable context storage in the session and arbitrary context management in the harness because we can't predict what specific context engineering will be required in future models. The interfaces push that context management into the harness, and only guarantee that the session is durable and available for interrogation.

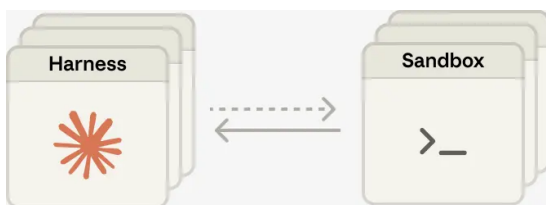
Many brains. Decoupling the brain from the hands solved one of our earliest customer complaints. When teams wanted Claude to work against resources in their own VPC, the only path was to peer their network with ours, because the container holding the harness assumed every resource sat next to it. Once the harness was no longer in the container, that assumption went away. The same change had a performance payoff. When we initially put the brain in a container, it meant that many brains required as many containers. For each brain, no inference could happen until that container was provisioned; every session paid the full container setup cost up front. Every session, even ones that would never touch the sandbox, had to clone the repo, boot the process, fetch pending events from our servers.

That dead time is expressed in time-to-first-token (TTFT), which measures how long a session waits between accepting work and producing its first response token. TTFT is the latency the user most acutely *feels*.

Decoupling the brain from the hands means that containers are provisioned by the brain via a tool call (`execute(name, input) → string`) only if they are needed. So a session that didn't need a container right away didn't wait for one. Inference could start as soon as the orchestration layer pulled pending events from the session log. Using this architecture, our p50 TTFT dropped roughly 60% and p95 dropped over 90%. Scaling to many brains just meant starting many stateless harnesses, and connecting them to hands only if needed.

Many hands. We also wanted the ability to connect each brain to many hands. In practice, this means Claude must reason about many execution environments and decide where to send work—a harder cognitive task than operating in a single shell. We started with the brain in a single container because earlier models weren't capable of this. As intelligence scaled, the single container became the limitation instead: when that container failed, we lost state for every hand that the brain was reaching into.

Decoupling the brain from the hands makes each hand a tool, `execute(name, input) → string`: a name and input go in, and a string is returned. That interface supports any custom tool, any MCP server, and our own tools. The harness doesn't know whether the sandbox is a container, a phone, or a Pokémon emulator. And because no hand is coupled to any brain, brains can pass hands to one another.



The challenge we faced is an old one: how to design a system for “programs as yet unthought of.” Operating systems have lasted decades by virtualizing the hardware into abstractions general enough for programs that didn't exist yet. With Managed Agents, we aimed to design a system that accommodates future harnesses, sandboxes, or other components around Claude.

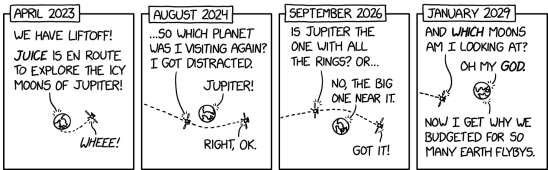
Managed Agents is a meta-harness in the same spirit, unopinionated about the *specific* harness that Claude will need in the future. Rather, it is a system with general interfaces that allow many different harnesses. For example, Claude Code is an excellent harness that we use widely across tasks. We've also shown that task-specific agent harnesses excel in narrow domains. Managed Agents can accommodate any of these, matching Claude's intelligence over time.

Meta-harness design means being opinionated about the interfaces around Claude: we expect that Claude will need the ability to manipulate state (the session) and perform computation (the sandbox). We also expect that Claude will require the ability to scale to many brains and many hands. We designed the interfaces so that these can be run reliably and securely over long time horizons. But we make no assumptions about the number or location of brains or hands that Claude will need.

Written by Lance Martin, Gabe Cemaj and Michael Cohen. Special thanks to the Agents API team and Jake Eaton for their contributions.

Jupiter Icy Moons Explorer

XKCD.COM · 29 SEP 2026 · [SOURCE](#)



"I did briefly visit Venus in August 2025, but I figured out the mistake on my own because it didn't have any moons."